Practical no 6: Creating classes and objects, using instances as arguments and return values, demonstrating inheritance, method overriding, data encapsulation and data hiding in Python.

Practical 1: Creating Classes and Objects

Question

Write a Python program to create a class named Student with attributes **name** and **roll number**. Create two objects of the class and display their details.

Code

```
# Creating a class


class Student:


    # Constructor
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll


    # Method to display details
    def display(self):
        print("Student Name :", self.name)
        print("Roll Number  :", self.roll)



# Creating objects
student1 = Student("Alice", 101)
student2 = Student("Bob", 102)


# Calling methods
student1.display()
print()
```

student2.display()

Sample Output

Student Name : Alice

Roll Number : 101

Student Name : Bob

Roll Number : 102

Practice

Create an Employee class with Employee ID, Name, and Salary.

Practical 2: Passing Objects as Arguments

Question

Write a Python program to pass an object of one class as an argument to another class.

Code

```
# Student class
class Student:

    def __init__(self, name):
        self.name = name



# College class
class College:

    def showStudent(self, student):
        print("Student Name :", student.name)



# Creating object
s1 = Student("Rahul")
```

```
# Passing object
college = College()
college.showStudent(s1)
```

Sample Output

```
Student Name : Rahul
```

Practice

Create a Book class and pass its object to a Library class.

Practical 3: Returning Objects from Methods

Question

Write a Python program to return an object from a method.

Code

```
class Student:

    def __init__(self, name):
        self.name = name



class StudentFactory:

    def createStudent(self):
        student = Student("Priya")
        return student



factory = StudentFactory()

student = factory.createStudent()

print("Student Name :", student.name)
```

Sample Output

Student Name : Priya

Practice

Create a CarFactory class that returns a Car object.

Practical 4: Inheritance

Question

Write a Python program to demonstrate single inheritance.

Code

```
# Parent class
class Person:

    def __init__(self, name):
        self.name = name

    def display(self):
        print("Name :", self.name)



# Child class
class Student(Person):

    def __init__(self, name, roll):
        super().__init__(name)
        self.roll = roll

    def show(self):
        self.display()
        print("Roll Number :", self.roll)



student = Student("Anjali", 101)
```

```
student.show()
```

Sample Output

```
Name : Anjali
Roll Number : 101
```

Practice

Create an Animal class and a Dog class that inherits from it.

Practical 5: Method Overriding

Question

Write a Python program to demonstrate method overriding.

Code

```
class Animal:

    def sound(self):
        print("Animals make sounds.")



class Dog(Animal):

    def sound(self):
        print("Dog barks.")



dog = Dog()

dog.sound()
```

Sample Output

```
Dog barks.
```

Practice

Create a Vehicle class and override the start() method in Car.

Practical 6: Data Encapsulation

Question

Write a Python program to demonstrate encapsulation using getter and setter methods.

Code

```
class BankAccount:

    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):

        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient Balance")

    def getBalance(self):
        print("Balance :", self.balance)



account = BankAccount()

account.deposit(5000)

account.withdraw(1200)

account.getBalance()
```

Sample Output

Balance : 3800

Practice

Create a Student class with methods to set and get marks.

Practical 7: Data Hiding

Question

Write a Python program to demonstrate data hiding using private variables.

Code

```
class Employee:

    def __init__(self):
        self.__salary = 50000

    def getSalary(self):
        return self.__salary

    def setSalary(self, salary):

        if salary > 0:
            self.__salary = salary



employee = Employee()

print("Salary :", employee.getSalary())

employee.setSalary(60000)

print("Updated Salary :", employee.getSalary())
```

Sample Output

Salary : 50000

Updated Salary : 60000